

UNIVERSIDAD DE LA CUENCA DEL PLATA

Facultad de Ingeniería, Tecnología y Arquitectura · Ingeniería en Sistemas de Información · Proyecto Final de Grado

Guía de comprobación del prototipo v1

Procedimiento que la cátedra aplica sobre el repositorio entregado · Instrumento de verificación de la dimensión 4

ISI-PFG-2026C2-AE2-v02 · Comisión A · Sede Posadas · Docente Titular: PosDr. Darío Ezequiel Díaz

La consigna de la AE2 invoca dos instrumentos de verificación específicos: la lista de control de requisitos y esta guía, que la cátedra ejecuta sobre el repositorio entregado. Se publica antes de la entrega para que el equipo pueda replicarla, que es la única manera razonable de llegar sin sorpresas.

⚠ LA CONDICIÓN MATERIAL, ENUNCIADA SIN RODEOS

Un esqueleto que no arranca, no compila o carece de instrucciones de ejecución **no se computa como entregado**, con independencia de la calidad del documento que lo acompañe. No es una dimensión de la rúbrica que pueda compensarse con otras: es un hecho del mundo, verificable por un tercero en una computadora que no es la del equipo, y del cual depende que el Producto principal pueda declararse completo.

1 · Qué se comprueba

La comprobación responde cuatro preguntas y ninguna se refiere a la belleza del código ni a la cantidad de funcionalidad implementada.

#	Pregunta	Qué la responde
1	¿El proyecto existe en el estado declarado?	La etiqueta <code>v1</code> existe y apunta a una revisión del repositorio entregado.
2	¿Puede ejecutarlo alguien que no lo escribió?	El archivo de lectura contiene instrucciones completas, y se sigue al pie de la letra, sin suplir pasos.
3	¿El recorrido atraviesa todas las capas?	El caso de uso vertical declarado va de la interfaz a la persistencia y vuelve, con una regla de negocio validada en el camino.
4	¿La integración continua está activa?	Existe el archivo de configuración del canal, corrió efectivamente, y su registro de corridas es accesible.

2 · El procedimiento, paso a paso

#	Qué hace la cátedra	Qué debe ocurrir
1	Clona el repositorio en una carpeta nueva de una máquina limpia, con la cuenta de colaboradora concedida.	El clonado se completa. Si el repositorio es privado y la cuenta no fue agregada, la comprobación se detiene aquí.
2	Se posiciona en la etiqueta v1 .	La etiqueta existe. Una rama llamada «v1» no es una etiqueta y no se acepta como tal.
3	Abre el archivo de lectura y lee la sección de requisitos previos.	Están declaradas las versiones de lenguaje, motor de base de datos y herramientas necesarias.
4	Instala dependencias con el comando que el archivo indica.	La instalación termina sin errores a partir del archivo de dependencias versionado en el repositorio.
5	Copia el archivo de variables de ejemplo y lo completa con valores locales.	El archivo de ejemplo existe, nombra todas las variables necesarias y ninguna contiene credenciales reales.
6	Crea el esquema de la base de datos con el guion o la migración provistos.	El esquema se crea de manera reproducible. Una base creada a mano, cuya estructura no está en el repositorio, no satisface este paso.
7	Arranca la aplicación con el comando indicado.	El proceso levanta y responde en la dirección declarada.
8	Ejecuta el caso de uso vertical declarado en el Instrumento 35.	El dato ingresa, la regla de negocio se aplica, el dato persiste, se recupera y se muestra.
9	Consulta el registro de corridas del canal de construcción.	Hay al menos una corrida exitosa con fecha anterior o igual a la de la entrega.

◆ LA REGLA QUE GOBIERNA LA COMPROBACIÓN

La cátedra no interpreta ni suple pasos faltantes. Si el proyecto necesita un comando que el archivo de lectura no menciona, ese comando no se ejecuta y el paso se registra como no satisfecho. La razón no es rigidez administrativa: un artefacto que sólo corre con conocimiento tácito de sus autores no es un artefacto entregable, y en noviembre el tribunal se encontrará con el mismo problema.

3 · Qué debe contener el archivo de lectura

El archivo `README.md` ubicado en la raíz del repositorio, o en `/src` si el equipo prefiere, contiene ocho secciones. El kit de repositorio publicado junto con esta guía trae el modelo completo.

§	Sección	Contenido mínimo
1	Identificación	Proyecto, equipo, integrantes, comisión y actividad a la que corresponde la etiqueta.
2	Qué hace este prototipo	El caso de uso vertical implementado, en dos oraciones, y qué decisión arquitectónica prueba.
3	Requisitos previos	Versiones exactas de lenguaje, motor de base de datos y herramientas. «Última versión» no es una versión.
4	Instalación	Comandos en el orden en que se ejecutan, copiables tal cual.
5	Configuración	Cómo se genera el archivo de variables a partir del de ejemplo y qué significa cada variable.
6	Ejecución y verificación	Cómo se arranca, en qué dirección responde y qué pasos concretos reproducen el caso de uso vertical, con el resultado esperado.
7	Estado del canal de construcción	Dónde se consulta el registro de corridas y qué verifica el canal.
8	Declaración de herramientas auxiliares	Herramienta, función y artefacto afectado, conforme al Protocolo de Uso Autorizado. Si no hubo uso, conviene consignarlo de manera expresa.

4 · La etiqueta v1

Una etiqueta fija una revisión concreta y permite que la cátedra evalúe exactamente lo que el equipo entregó, aunque el repositorio siga avanzando después. Se crea una sola vez, con anotación, y se publica.

```
git add -A
git commit -m "feat: caso de uso vertical completo, esqueleto arquitectónico v1"
git tag -a v1 -m "Prototipo v1 · AE2 · esqueleto arquitectónico ejecutable"
git push origin main
git push origin v1
```

Publicada la etiqueta, conviene comprobar que existe del lado del servidor: `git ls-remote --tags origin` debe listarla. Si después de etiquetar el equipo corrige algo, se crea `v1.1` y se informa en la entrega cuál rige; no se mueve la etiqueta ya publicada.

⚠ MODIFICACIONES POSTERIORES A LA ENTREGA

Toda modificación posterior a las 23:59 del 1.º de octubre se presenta como versión nueva dentro de la Ventana, con su propia etiqueta, y no como reemplazo silencioso de lo entregado. La corrección de la AE1 ya observó un caso de actualización posterior a la carga; lo que se evalúa es lo que estaba en el momento del cierre.

5 · Integración continua: el mínimo aceptable

El canal no necesita ser sofisticado. Tres pasos satisfacen la exigencia: instalar dependencias, construir el proyecto y ejecutar al menos una prueba automatizada que verifique el criterio de aceptación del requisito implementado en el recorrido vertical. Ese único caso de prueba tiene un efecto adicional que se agradece en octubre: obliga a que el criterio de aceptación esté escrito con la precisión suficiente para poder programarse.

Qué se verifica	Cómo se acredita
Existencia del canal	Archivo de configuración versionado en el repositorio: <code>.github/workflows/ci.yml</code> o <code>.gitlab-ci.yml</code> .
Ejecución efectiva	Registro de corridas con al menos una exitosa, con fecha. Un archivo de configuración que nunca corrió acredita una intención, no un canal.
Accesibilidad	El registro debe poder consultarse con la cuenta que la cátedra tiene concedida sobre el repositorio.
Contenido mínimo	Instalación de dependencias, construcción y al menos una prueba automatizada asociada a un criterio de aceptación del catálogo.

6 · Las seis causas de fallo más frecuentes

Síntoma en la comprobación	Causa habitual	Cómo se previene
Falta un módulo o paquete	Dependencia instalada globalmente en la máquina del autor y no declarada en el proyecto.	Versionar el archivo de dependencias y probar la instalación en una carpeta nueva.
Error de conexión a la base	Variables de entorno presentes en el equipo y ausentes del archivo de ejemplo.	Mantener el archivo de ejemplo con todas las variables, con valores ficticios.
Tabla o columna inexistente	Esquema creado a mano, sin guion ni migración en el repositorio.	Versionar el guion de creación del esquema y probarlo sobre una base vacía.
El proceso arranca y la página no carga	Dirección o puerto distintos de los declarados en el archivo de lectura.	Verificar la sección de ejecución después de escribirla, no antes.
El repositorio no se puede clonar	Repositorio privado sin la cuenta de la cátedra como colaboradora.	Conceder el acceso al crear el repositorio, no la víspera de la entrega.
La etiqueta no existe	Etiqueta creada localmente y no publicada.	<code>git push origin v1</code> y comprobación con <code>git ls-remote --tags origin</code> .

7 · Autocomprobación previa

La realiza un integrante que no construyó el esqueleto, en una computadora distinta de la del autor, siguiendo únicamente el archivo de lectura y sin agregar pasos de memoria. Si hace falta agregar alguno, ese paso se escribe en el archivo y la prueba se repite desde el comienzo. El registro de esta autocomprobación se consigna en el Instrumento 35 y se aloja en `/src`.

≡ EN SÍNTESIS

- La etiqueta `v1` existe, está publicada y apunta a lo que se entrega.
- El archivo de lectura permite ejecutar sin conocimiento previo del proyecto.
- El recorrido vertical atraviesa interfaz, lógica, persistencia y retorno, y valida una regla de negocio.
- El canal de construcción corrió, con registro accesible y fecha anterior o igual a la de la entrega.
- La prueba de clonado la hizo alguien que no escribió el código, en otra máquina.

8 · Registro de la comprobación

La cátedra completa esta grilla por equipo y la incorpora a la devolución escrita. Se publica para que el equipo sepa exactamente qué se consignará sobre su artefacto.

Verificación	Resultado	Observación
Repositorio accesible con la cuenta concedida	Sí / No	
Etiqueta v1 existente y publicada	Sí / No	
Archivo de lectura con las ocho secciones	Completo / Parcial / Ausente	
Instalación de dependencias sin pasos suplidos	Sí / No	
Creación reproducible del esquema	Sí / No	
Arranque de la aplicación	Sí / No	
Recorrido vertical ejecutado de punta a punta	Sí / No	
Regla de negocio validada en la capa de lógica	Sí / No	
Canal de construcción con corrida exitosa accesible	Sí / No	
Cómputo del artefacto	Computado / No computado	